

SE3002 Assignment 02 Report

SE3002 Assignment 02 — Structural Testing of PX4-Autopilot v1.17.0

1. Group details and baseline

Student	Name	Roll number	Primary area	
S1	Taimoor Shaukat	24i3015	environment, baseline, coverage, gaps	
S2	Muhammad Bilal Tahir	24i3166	DataValidator + DataValidatorGroup, MC/DC	
S3	Ali	24i3158	FailureDetector/Injector, findings, report	

Section B, Course SE3002 Software Quality Engineering, Assignment 02.

Baseline: tag v1.17.0, built from commit d6f12ad1c4f70ad3230afd7d86e971421e02fef4 (this is the commit the tag points to via v1.17.0^{commit}, not the annotated tag object hash a5eb12d2ab591251faa009f76b2685b8cc64405d, which is a distinct git object — citing the tag object instead of the commit it resolves to is a documented mistake to avoid, see [work/explain/P01.md](#)). Work was carried out on branch sqe-a2, with merge-base with v1.17.0 confirmed equal to the required commit.

2. Local environment and setup evidence

All tooling versions were captured directly, not assumed: Ubuntu 24.04.4 LTS, x86_64, 13th Gen Intel Core i7-1355U (12 cores), 30 GiB RAM, kernel 7.0.0-34-generic. Compiler: GCC/G++ 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1); cmake 4.4.3; ninja 1.13.2; GNU Make 4.3; Python 3.12.3; lcov/genhtml 2.0-1; gcov 13.3.0; git 2.43.0. No Clang was installed or required — the macOS Coverage-build flag fix documented in the project's reference material does not apply to this Linux/GCC path.

PX4's own setup script (Tools/setup/ubuntu.sh) installs its toolchain via apt-get, which requires sudo; this environment had no non-interactive sudo available (sudo -n true failed). Rather than block on that, every missing tool was installed user-locally instead, with no elevation: cmake/ninja via pip install --user; lcov/genhtml 2.0 built from the upstream lcov source with make install PREFIX=~/.local; the one missing Perl module (DateTime) installed into a user-local local::lib/cpan

environment. gcc/g++/gcov and python3 were already present system-wide. `~/ .bashrc` was updated to put `~/ .local/bin` on PATH. This is a legitimate, reproducible deviation from the documented sudo-based path, not a shortcut around a requirement; a reader with working sudo access can use the standard `Tools/setup/ubuntu.sh` script instead (Appendix A documents both).

Build/test commands used throughout: `tools/sqe_run_tests.sh build|run|probes|shuffle` and `tools/sqe_coverage.sh build|baseline|student|final` (wrappers around `cmake --build build/px4_sitl_test` and `ctest`), never upstream `make tests_coverage` (which runs `make clean`, omits branch data, and silently ignores lcov mismatches).

Baseline `make tests` (upstream suite, unmodified source, before any student test code existed): **100% tests passed, 147 of 147**, total test time 45.73s. No pre-existing failures were observed at baseline.

3. Repository analysis and scope selection record

Four areas (A–D) were selected after re-measuring candidates directly against the live v1.17.0 source (not copied from pre-derived reference material) via grep-based decision/compound-decision counts and an upstream-gtest registration search (119 `px4_add_unit_gtest/px4_add_functional_gtest` lines, none matching `data_validator` or `failure_detector`).

Area A — DataValidator.cpp (`src/modules/sensors/data_validator/`): tracks one sensor instance’s value stream and derives a 0.0–1.0 confidence from timeout, staleness and error-density/count, feeding the Area B voter. 155 lines, 17 decision points, 0 compound decisions. No params, no uORB; all inputs are explicit `put()`/`confidence()` arguments. Test level: unit (`px4_add_unit_gtest`), since no `DEFINE_PARAMETERS/uORB` subscription/runtime-service construction occurs outside the diagnostic `print()` method.

Area B — DataValidatorGroup.cpp (MC/DC component): owns a linked list of `DataValidators`, ranks by confidence × priority, selects the “best” sensor and classifies failover events. 347 lines, 42 decision points, **7 compound decisions** — the densest unit-level candidate. Confirmed call sites: `modules/sensors/voted_sensors_update.cpp` (IMU accel/gyro voting) and `modules/sensors/vehicle_magnetometer/VehicleMagnetometer.cpp` (magnetometer voting) — this logic directly decides which sensor instance feeds the state estimator. Test level: unit, same reasoning as A.

Area C — FailureDetector.cpp: detects attitude-envelope exceedance, ESC/telemetry faults, motor under-current, imbalanced-propeller vibration and external-ATS signals, aggregating into Commander’s failsafe status bitmask. 354 lines, 46 decision points, **15 compound decisions** — the largest and most logically dense area. 8 params read at construction (`FailureDetector.hpp`: 129–138), 6 uORB subscriptions. Confirmed call site: `modules/commander/Commander.cpp`: 1857. Test level: functional (`px4_add_functional_gtest`) — params are read once at construction, requiring the set-params-then-construct ordering; `gtest_functional_main` initializes uORB/params.

Area D — FailureInjector.cpp: reads `SYS_FAILURE_EN` and injected `vehicle_commands` to simulate motor failures for in-the-loop testing of Area C. 134 lines, 17 decision points, 4 compound decisions. Test level: functional, same reasoning as C.

Together A–D cover ~990 source lines, ~122 decisions, ~26 compound decisions across two safety chains (sensor redundancy A+B; failure detection C+D), both pure control/business logic with zero existing upstream GTest coverage.

Exclusions (file-spanning, D-004, flagged HUMAN-DECISION for team review): trivial header-only accessors (single-statement returns, no branches); the orphaned `src/modules/sensors/data_validator/tests/` directory (confirmed dead — `data_validator/CMakeLists.txt` never adds it, observation O-01); `print()` diagnostic-dump methods kept in scope for statement/branch coverage but excluded from MC/DC derivation as non-business-logic output.

Candidates rejected (re-measured against live source, none selected for the final scope): `lib/battery/battery.cpp` (stretch candidate, not needed); `land_detector/*` (heavy module-lifecycle harness, better suited to SITL); `HealthAndArmingChecks/checks/batteryCheck.cpp` (tightly coupled to shared check framework); `commander/failsafe/{failsafe, framework}.cpp` (upstream `failsafe_test.cpp` already exists, too large for the time box); `manual_control/ManualControlSelector.cpp` and `navigator/GeofenceBreachAvoidance` (already upstream-tested); `lib/hysteresis/hysteresis.cpp` (already upstream-tested, used only as a dependency of Area C); `commander/Safety.cpp` and `commander/UserModeIntention.cpp` (too small a decision surface). Re-verification corrected two REF_02 entries during candidate re-measurement: `Safety.cpp` and `UserModeIntention.cpp` were listed there as upstream-tested, but no file-specific gtest registration was found for either — the rejection verdict is unaffected, but the stated reason was corrected for accuracy.

4. Testing approach

Decisions were derived per-file from the live v1.17.0 source into per-area inventories (`work/basis/INVENTORY_DataValidator.md`, `INVENTORY_DataValidatorGroup.md`, `INVENTORY_FailureDetector_Injector.md`) plus three control-flow sketches (`CFG_DataValidator_confidence.md`, `CFG_DataValidatorGroup_get_best.md`, `CFG_FailureDetector_updateMotorStatus.md`) before any test code was written. A setup map (`work/basis/SETUP_MAP.md`) records, per area, exactly how each decision is controlled: A/B take every input as an explicit function argument (`put()`, `confidence()`, `get_best()`); C/D require the ordering `params-set` → `object-constructed` → `uORB-published` → `update()`-called, since `DEFINE_PARAMETERS` is read once at construction and `uORB` has latest-sample (not queued) semantics for single-instance topics.

Oracles follow SPEC_02 §3's allowed sources: return values, getters, published `uORB` messages, flags/bitmasks and counters, derived from code comments/headers/ documented library contracts or independently-derived arithmetic — never the production expression re-copied into the test (a circular oracle) and never “does not crash” alone. Several tests are explicitly **characterization tests** (asserting actual behaviour where the specification is silent/ambiguous, labelled `characterization (F-nn)` in their header comment per SPEC_02 §3): DV07 (all-NaN stream), DV19

(`print()`'s side effect), DV20 (timestamp-0 sentinel) and FD33 (motor-mask-survives-disarm). These are reported as characterizations, not specification-backed oracles, and are cross-referenced to their corresponding findings in §7.

Independence and repeatability were demonstrated per binary with `--gtest_shuffle` and repeat counts of 5 for the two unit binaries and 10 for the three functional binaries; every shuffle run reports zero failures: `unit-SqeDataValidator` 20/20 PASSED ×5, `unit-SqeDataValidatorGroup` 34/34 PASSED ×5, `functional-SqeFailureDetector` 34/34 PASSED ×10, `functional-SqeFailureDetectorImu` 7/7 PASSED ×10, `functional-SqeFailureInjector` 13/13 PASSED ×10. Each test also passes individually under `--gtest_filter`.

The full 1:1 test-to-decision mapping (116 inventory rows: 20 SQE-DV, 12 SQE-DVG, 22 SQE-DVG-MC, 36 SQE-FD, 8 SQE-FDI, 13 SQE-FI, 5 SQE-PRB probes) is not duplicated here — see `work/inventory/test_inventory.csv` and the “Test Inventory” sheet of the submitted workbook (`deliverables/24i3015_24i3166_24i3158_B.xlsx`).

5. MC/DC component selection and interpretation

`DataValidatorGroup` was chosen for the assignment's one required MC/DC component because, among the four scoped areas, it combines the highest compound-decision density at the unit level (5 genuinely compound Boolean decisions, 2–7 atomic conditions each, pure functions of explicit arguments/internal state with no params/uORB/hidden I/O) with the clearest safety argument: its `get_best()` decides which physical sensor feeds the EKF state estimator, and `failover_index()`/`failover_state()` report when and why a failover occurred. `FailureDetector` has more compound decisions in absolute count but is covered to decision/branch level only, per the assignment's “one justified component” requirement (D-003).

The 5 decisions: DVG-D13 (`get_best()` candidate-switch test, L187-190, 7 conditions A–G); DVG-D14 (“did best change”, L203, 3 conditions H/I/J); DVG-D15 (“real failsafe vs priority preference”, L207-208, 3 conditions K/L/M); DVG-D32/DVG-D34 (`failover_index()/failover_state()`, L282-283/L301-302, identical 3-condition pattern P/Q/R). 29 target-evaluation rows across 22 distinct test IDs implement these five decisions with unique-cause independence pairs throughout — no masking was needed anywhere in the matrix, a result worth noting as it indicates the short-circuit structure did not force a weaker masking-MC/DC argument.

Three worked examples. The **E pair** (DVG-D13, `fabsf(confidence_max_confidence)<0.01f`): SQE-DVG-MC-01 seeds sensor 0 at confidence 0.95/priority 50, then introduces sensor 1 at confidence 0.95/priority 75 (`|c-m|=0<0.01f`, `E=True`); its independence pair SQE-DVG-MC-03 changes only the candidate's confidence to 0.93 (`|0.93-0.95|=0.02f≥0.01f`, `E=False`), holding every other condition's evaluated value fixed, and the decision outcome flips (switch vs no switch).

The **K pair** (DVG-D15, `pre_check_prio != -1`): when `K=False` (no prior “best” was ever seeded), the outer `if (best != nullptr)` guard at L213 is structurally forced reachable-but-always-True by a separate dominance proof (see §6 G-03), and `_curr_best<0` at L219 makes the `true_failsafe` value irrelevant to any getter — so the `K=False` row (SQE-DVG-MC-21) cannot be distinguished from its True counterpart (SQE-DVG-MC-01) by any API return value. This row is classified **O3** (structural

evidence only): line 210 (`true_failsafe = false;`) shows 0 hits for MC21 versus 1 hit for MC01 in their respective per-test `.info` captures, which is the only available proof the test reached its intended branch.

The **P pair** (DVG-D32/D34, shared by `failover_index()/failover_state()`): SQE-DVG-MC-13 and SQE-DVG-MC-25 form the P pair; SQE-DVG-MC-25 specifically constructs the “put with timestamp 0” scenario (`DataValidator.cpp`’s `timestamp-0` sentinel, F-12) to drive the validator’s `used()` state to `False`, which flips `P = next->used()` while Q and R’s evaluated values are held fixed.

Three condition combinations in DVG-D13 and one in DVG-D14 were determined **infeasible (with proof)**, not simply omitted: $B=T \wedge G=F$ and $F=T \wedge D=F$ are excluded because $B \Rightarrow G$ and $F \Rightarrow D$ are real implications of the underlying float/priority comparisons; $A=T \wedge B=T \wedge C=F$ is excluded because $(A \wedge B) \Rightarrow C$; and DVG-D14’s $H=F \wedge I=F \wedge J=F$ is excluded by an algebraic trace showing $J=F$ forces $I=T$. A related float-precision characteristic (F-06: the 1% tie-break’s $1.0f - d/100.0f$ is not exact at integer d , confirmed by direct float32 arithmetic, e.g. $d=10 \rightarrow 0.89999998f$) is documented but deliberately not exercised by any of the 12 chosen D13 vectors, each kept ≥ 0.01 – 0.02 away from the boundary so the suite’s PASS/FAIL results are stable across runs.

The matrix was independently re-derived by hand and cross-checked with a standalone Python mirror of the exact C++ operator/short-circuit order; `tools/sqe_mcdc_check.py` reports ALL DECISIONS COMPLETE for all 5 decisions, confirming every condition has both a True-outcome and False-outcome evaluation with a traced unique-cause independence pair.

6. Coverage analysis (baseline vs final, raw vs feasible)

Baseline (upstream tests only, before any student test code): `DataValidator.cpp` 43/58 lines (74.1%), 21/30 branches (70.0%); `DataValidatorGroup.cpp` 101/155 lines (65.2%), 54/116 branches (46.6%); `FailureDetector.cpp` and `FailureInjector.cpp` both 0/153 and 0/62 lines (0.0%/0.0%) and 0 branches, since no upstream test starts Commander.

Final (full student suite, 5 binaries): `DataValidator.cpp` 58/58 lines (100.0%), 30/30 branches (100.0%); `DataValidatorGroup.cpp` 154/155 lines (99.4%), 110/116 branches (94.8%); `FailureDetector.cpp` 153/153 lines (100.0%), 189/254 branches (74.4%); `FailureInjector.cpp` 62/62 lines (100.0%), 47/57 branches (82.5%). Total across the scope: 427/428 lines (99.8%), 376/457 branches (82.3%). A separate student-only capture (excluding the upstream suite entirely) produced identical numbers, confirming the student suite alone drives all of the scope’s measured coverage, not an interaction with pre-existing upstream tests.

This is `gcov/lcov`’s **raw** tool number. Per the course’s stated `gcov` semantics, “branch coverage” here is condition-level coverage of *evaluated* operands for short-circuit `&&/||` expressions (one branch pair per short-circuit operand), not full MC/DC — the MC/DC matrix in §5 is the independent, stronger claim for `DataValidatorGroup` specifically.

Coverage was reached in three iterations after the IT-0 baseline. **IT-1** (first full capture with all 5 binaries) investigated every uncovered item via raw `gcov -b -c` re-derivation (not just `lcov`’s BRDA summary) and classified most as tool-artefact or infeasible, while identifying 3 genuine missing-test gaps (FD-D25, FD-D38, FD-D41). **IT-2** closed those 3 gaps with 3 new tests (SQE-FDI-08, SQE-FD-36, SQE-FD-37), raising

FailureDetector.cpp branches from 185/254 (72.8%) to 189/254 (74.4%) with no other file's numbers changing. **IT-3** was a confirmatory re-run (no test changes) that reproduced IT-2's numbers byte-for-byte, ruling out leftover .gcda state as an artefact of the debugging session.

Raw vs feasible. work/GAPS.md documents 7 clusters (G-01...G-07) covering every item left uncovered after IT-3, each with location, class (environment-limited / infeasible-with-proof / tool-artefact) and a written proof or reproduction path — none excluded merely to raise a percentage, and no LCOV_EXCL_* marker was added to production code. Two items are **infeasible with proof**: G-03 (DataValidatorGroup.cpp:213's `best != nullptr` False side — the same code-flow dominance argument used for DVG-D15's `K=False` row in §5 shows `best` is unconditionally non-null whenever this guard is reached) and G-06 (FailureInjector.cpp:43-44's `param_get()` failure branch — `SYS_FAILURE_EN` is a compile-time-registered parameter, so `param_find()` on this exact literal name cannot fail in a successfully-linked binary). G-07 is infeasible specifically in a single-threaded test environment (FailureDetector.cpp:194,222's `copy()` False immediately after `.updated()` True — traced through the real uORB DeviceNode/Subscription implementation to show `.updated()⇒data≠nullptr` in the absence of concurrent writers). One item is **environment-limited**: G-01 (DataValidatorGroup.cpp:88's `-fcheck-new` null-check — real on a non-throwing NuttX allocator, dead code on this build's throwing operator `new` ABI; not reached without a dedicated allocator-fault build, not attempted this session). Three are **tool-artefact**: G-02 (`delete`'s compiler-emitted null-guard on a loop-invariant-guaranteed-non-null pointer), G-04 (gcov's documented line-attribution collapse of 6 ternary conditions in `print()`'s multi-argument `PX4_INFO_RAW` call onto one source line — independently reproduced with a standalone minimal repro under the same coverage flags) and G-05 (GCC's per-call exception-unwind edges across FailureDetector/FailureInjector, confirmed via `gcov -b -c` showing taken 0 (throw) on every entry, for library calls that do not throw by PX4 convention). Excluding these 7 clusters' items from the denominator (the “feasible” view) would bring both files' reachable branch coverage close to 100%, but this report states the raw number (above) as the primary figure and treats the feasible view as the explicitly-justified gap list in §8, per SPEC_03/P10's “report raw, then feasible” rule — a single feasible percentage figure is not synthesized here beyond what work/GAPS.md itself computes, to avoid an unverified rounding claim.

Annotated excerpts (from evidence/coverage/final/html/src/modules/sensors/data_validator/DataValidatorGroup.cpp.gcov.html): at L187-190 (DVG-D13's 7-condition expression), the gcov branch-pair annotations show, e.g., branch 0 (A) taken 454 times and branch 1 (B, only evaluated when A is false) taken 29 times at L187, with every branch pair on these four lines showing a non-zero “taken” count for both outcomes — consistent with the MC/DC matrix's claim of full True/False coverage for all 7 conditions. At L207-213 (DVG-D15's guard and the nested `best != nullptr` check), L207's K/L/M branches show non-zero counts on both sides, but L213's second branch (`best == nullptr`) shows tlaUNC/“Branch 1 was not taken” with 0 hits for both sides where L210 (`true_failsafe = false;`) is executed exactly 2 times total — consistent with G-03's infeasibility proof (the False side of L213 is structurally unreachable) and with the K pair's O3 classification in §5.

7. Findings

All 15 candidate findings from the project's pre-derived register were independently re-investigated this session against the live source and the actual executed test/probe results; **13 confirmed, 0 rejected, 1 latent and not runtime-verified** (F-13). Three findings (F-09, F-10, F-11) initially required a specification-level oracle judgment the evidence alone could not settle; these have since been resolved with documented reasoning (F-09 and F-10 as confirmed defects, F-11 as a specification ambiguity rather than a defect — full reasoning in `work/FINDINGS.md`), on the explicit authority of the team member directing this session, and remain open for the whole team's review before the viva. That zero candidates were rejected is reported as a genuine outcome, not evidence of padding: every REF_07 candidate's underlying code-level claim held up under independent re-reading of the cited file:line this session.

Confirmed defects / observations (title — location — test ID — severity; full reproduction/expected/actual in `work/findings/REPORT_BLOCKS.md`, not duplicated here): - **F-01** `best != nullptr` guard (`DataValidatorGroup.cpp:213`) is dead code in its `False` branch — structural proof only, no severity (coverage-measurement artefact, cross-referenced to G-03 in §6/§8). - **F-02** `get_best()` returns a non-null stale-data pointer when `*index == -1` signals total sensor failure, contradicting the header's "array of best values" phrasing — SQE-DVG-11, SQE-DVG-MC-19 confirm; SQE-PRB-03 (disabled probe) fails under the stricter `nullptr` oracle. Latent: both production callers discard the return value and use only `*index`. - **F-03** `DataValidatorGroup(0) + add_new_validator()` dereferences a null `_last` and crashes (confirmed via `EXPECT_DEATH`, SQE-DVG-10) — unreachable from production (all 3 real construction sites use `{1}`). - **F-04** the stale-value counter is shared across all 3 axes rather than per-axis, so a padded scalar sensor (e.g. `airspeed`, `sensors.cpp:285`) reaches "stale" roughly 3× faster than a true 3-axis signal — SQE-DV-05, documented design trade-off. - **F-05** `DataValidator::print()` is not read-only — its internal `confidence()` call can set `ERROR_FLAG_TIMEOUT` as a side effect of printing — SQE-DV-19 (characterization). - **F-06** the 1% confidence tie-break is float-rounding/optimisation-level-sensitive (confirmed by direct float32 arithmetic) — not exercised by the submitted suite's deliberately boundary-avoiding vectors; treated separately as an untested-interaction risk in §8. - **F-07a/F-07b** an unclamped ESC-count loop and an unguarded shift exponent in `FailureInjector::manipulateEscStatus` — confirmed by static source analysis only; the corresponding probes (SQE-PRB-05/06) run as inert `SUCCEED()` placeholders in this session's non-sanitizer build, since the undefined behaviour is only detectable under ASan/UBSan, which was not built this session (risk/time judgment, documented in `work/FINDINGS.md`'s "Sanitizer evidence" note). - **F-08** the `FAILURE_TYPE_WRONG` log message uses a 0-based motor index while every sibling case uses 1-based — cosmetic, confirmed by direct source read, no dedicated capturing test written (R4 bars manufacturing an assertion to pad the finding). - **F-09 (confirmed defect)** a stream of only non-finite samples reports `confidence()==1.0/state()==NO_ERROR`, contradicting the class's own stated purpose of identifying anomalies — SQE-DV-07 (characterization), SQE-PRB-01 (now the approved stricter oracle) fails against the as-shipped code, as expected of a confirmed-defect probe. - **F-10 (confirmed defect)** `_error_density == ERROR_DENSITY_WINDOW` exactly yields `confidence()==0` with no flag set (`>` not `>=` at `DataValidator.cpp:125`) — an internal contradiction between two outputs of the same object for the same instant, not a boundary-placement question. SQE-DV-15, SQE-PRB-02 (now approved) fails as expected; consumer impact confirmed at `voted_sensors_update.cpp:419`. - **F-11 (specification ambiguity, not a**

defect) an equal-priority, confidence-only sensor switch increments `failover_count()` even though the source’s own comment frames the guard as distinguishing “a real failsafe” from “a priority preference” — SQE-DVG-MC-04, SQE-DVG-MC-07 confirm the current, self-consistent behaviour. Unlike F-09/F-10 this has no internal contradiction; resolving it requires knowing what the counter is used for downstream, which this session could not determine from source alone. - **F-12** timestamp 0 is an implicit “no data” sentinel in both `put()` and `confidence()` — SQE-DV-20, SQE-DVG-MC-25 confirm; not reachable in the real system since `hrt_absolute_time()` is never 0 after boot. - **F-15** `FailureDetector`’s disarm-reset block clears the under-current mask but never the timed-out mask, so a previously-timed-out motor still reports failed in `getMotorFailures()` after a clean disarm — SQE-FD-33 (characterization).

Two citation corrections to the pre-derived register were made and documented rather than silently fixed: F-02’s originally-cited evidence test (“MC18”) is a stable no-failover control row with no bearing on the finding — the real evidence is MC19; F-12’s originally-cited test (“MC17”) does not exist in the actual suite — the real DVG-level evidence is MC25. Neither correction changes the finding itself.

Rejected candidates: none (0 of 15). **Downgraded:** F-13 (allocation-failure robustness in `DataValidatorGroup`’s constructor/add_new_validator()) — the code shape (no null-safe handling reachable on this platform) is confirmed by reading the source, but the claimed runtime crash was not independently reproduced this session (no AF*-series test exists in the submitted suite, and the pre-derived register’s own “pre-verified” claim was made outside the PX4 GTest suite) — reported as latent, not a confirmed runtime defect.

8. Gaps, limitations, residual risk, improvements

The 7 coverage-gap clusters (G-01...G-07, §6) are the primary remaining structural gaps; each has a written proof or reproduction path in `work/GAPS.md` and is not repeated here. In summary: 1 environment-limited (G-01, NuttX non-throwing-allocator null-check, unreachable under this build’s throwing operator `new`), 2 infeasible-with-proof items plus a single-threaded-specific infeasibility (G-03, G-06, G-07), and 3 tool-artefact items (G-02, G-04, G-05, all independently reproduced with standalone minimal repros or raw `gcov -b -c` output, not merely asserted).

Environment limits. All evidence in this report comes from a single compiler/OS pairing — GCC 13.3.0 on Ubuntu 24.04 — with no cross-compiler build and no verification against an actual NuttX target. This matters concretely for G-01: the `-fcheck-new` null-check at `DataValidatorGroup.cpp:88` is dead code on this platform’s throwing-operator `new` ABI but is documented as real flight code for targets with a non-throwing (NuttX) allocator configuration — this report cannot claim that code path was exercised on any platform where it is live.

No SITL/HITL. This is a deliberate scope decision (§3, `work/scope/CANDIDATES.md` §4), not an oversight: none of the four scoped files touch drivers, the work-queue scheduler, module start/stop lifecycle or simulator dynamics to reach their decisions, so unit/functional GTest levels reach every decision deterministically without SITL’s added nondeterminism and wall-clock cost. The consequence is that this report says nothing about hardware timing, real sensor-driver interaction, or the actual multi-process/multi-

thread behaviour of `voted_sensors_update.cpp/Commander.cpp` calling into the scoped files — those call sites are confirmed to exist (§3) but their real runtime interaction with the scoped logic was never executed.

F-06 as an untested-interaction risk. The 1%-confidence tie-break’s float-rounding/optimisation-level sensitivity (§5, §7) is a specific, named risk that the coverage and MC/DC results in this report do not bound: the MC/DC matrix’s 12 D13 vectors were deliberately kept away from this boundary for test stability, which means the suite provides no evidence about behaviour exactly at that boundary under a different optimisation level (e.g. a -02/-03 flight build) than the -00 Coverage build used throughout this session.

Concrete improvements (not implemented this session, listed per SPEC_06 §8’s guidance): (1) a dedicated allocation-fault-injection build (SPEC_02 §9’s documented GCC/-fcheck-new technique, in its own CMake target) to close G-01 and give F-13 runtime evidence; (2) an ASan/UBSan rebuild to runtime-confirm F-07a/F-07b rather than leave them static-analysis-only; (3) a second optimisation-level build (-02) to directly measure F-06’s adjacent-step divergence instead of relying on hand float32 arithmetic; (4) dependency-level testability improvements such as splitting `print()`’s multi-ternary `PX4_INF0_RAW` call (G-04) into separate statements, which — if ever made as a production change outside this assignment’s R2 constraint — would let gcov attribute branches correctly per ternary.

9. Final quality judgment

The structural evidence in this report supports specific, bounded claims about the analysed scope — four files, `DataValidator.cpp`, `DataValidatorGroup.cpp`, `FailureDetector.cpp` and `FailureInjector.cpp` — and nothing broader. Measured raw coverage reached 99.8% lines and 82.3% branches across the scope, with `DataValidator.cpp` and `DataValidatorGroup.cpp` at 100.0%/100.0% and 99.4%/94.8% respectively, and every remaining uncovered item is classified with a written proof as infeasible, environment-limited, or a tool-measurement artefact rather than silently dropped. On `DataValidatorGroup`’s five redundancy-selection decisions, full MC/DC was achieved using unique-cause independence pairs throughout (no masking needed anywhere), with every condition’s True and False outcomes checked against an exact oracle — return values, state getters, or, for one structurally-unobservable row, documented per-test structural evidence. This combination gives high confidence that every implemented decision outcome in the scope, including each evaluated condition inside `DataValidatorGroup`’s compound expressions, was exercised at least once with a checked oracle.

That confidence does not extend to claims this evidence cannot support. Two of the three findings requiring a specification-level oracle decision (F-09, F-10) are now classified as confirmed defects, and the third (F-11) as a specification ambiguity rather than a defect, decided this session on the submitting team member’s authority rather than full team consensus — the corresponding probe tests fail by design against these confirmed/stricter readings and the team should independently review this reasoning before relying on it. The NuttX non-throwing-allocator path (G-01), hardware timing, and the real multi-threaded/multi-process interaction of the scoped files with their actual callers (`VotedSensorsUpdate`, `Commander`, `VehicleMagnetometer`) were never executed — only

the confirmed call sites were read, not exercised end-to-end. F-06's float-rounding sensitivity at the 1% confidence boundary is real but deliberately unexercised by the submitted test vectors.

Condition-level branch coverage plus full MC/DC on the voter's redundancy-selection logic together mean each implemented decision outcome was exercised with a checked oracle — this is not the same claim as “the requirements are complete” or “the code is correct for all inputs,” and this report does not make either of those claims.

What limits any broader conclusion: all testing was at unit/functional GTest isolation from the real sensor/estimator pipeline, on a single compiler and OS (GCC 13.3.0, Ubuntu 24.04), with no SITL or hardware-in-the-loop run. No claim is made, or should be inferred, about PX4 as a whole, about production-build (-02/-03) behaviour, or about any file outside the analysed scope.

10. AI-assistance record

AI (Claude Code) was used materially across all phases of this assignment: repository navigation and source verification (re-reading production code directly rather than trusting pre-derived reference material before every derivation), build/coverage-pipeline troubleshooting, test scaffolding and implementation (109 active student-authored GTest cases plus 5 disabled probes across 5 binaries), MC/DC derivation and independent cross-checking, coverage-gap investigation and classification, findings investigation against the live source, workbook generation, and this report's writing/synthesis.

Verification practice followed throughout: every test traces to a specific decision ID checked by `tools/sqe_trace_check.py` (clean, 0 errors/warnings after two tooling fixes documented in D-010); every MC/DC pair was independently recomputed by hand and cross-checked against a standalone Python mirror of the exact C++ operator/short-circuit order; every coverage number and finding in this report was re-read from its evidence file in this session rather than carried over from memory of an earlier phase. No test assertion was weakened to make a test pass (R4); the one test-design error found (SQE-FD-36/37's first draft, wrong assumption about ESC failure-mask bit layout) was corrected by rewriting the test before any coverage capture used it, not by loosening the oracle.

Key assumptions introduced and how they were checked: none were taken on faith — each classification in `work/GAPS.md` and `work/FINDINGS.md` required independently reading the cited production file:line and, in several cases, tracing further into PX4/uORB library internals (`uORBDeviceNode.cpp/.hpp`, `Subscription.hpp`) to construct a real proof rather than assert a claim. One explicit, honestly-reported assumption in the process itself: gate approvals (G00–G12, and this report's own G13) were self-approved autonomously under the user's standing session-start authorization rather than by a human reviewer; `work/STATUS.md` marks each as AUTO-APPROVED. The three findings requiring a specification-level oracle decision (F-09, F-10, F-11) were likewise resolved autonomously, on the submitting team member's standing authority, not by full team deliberation — this is flagged explicitly rather than presented as a settled human consensus, and the team should review that reasoning (`work/FINDINGS.md`) before the viva. The team takes responsibility for all targets, expected results, tests and conclusions in this submission.

Appendix A — Reproduction commands

Clone and baseline (work/explain/P01.md):

```
git clone --branch v1.17.0 --recursive <px4-remote> PX4-Autopilot
cd PX4-Autopilot && git rev-parse 'v1.17.0^{commit}' # must print
d6f12ad1c4f70ad3230afd7d86e971421e02fef4
git checkout -b sqe-a2
git submodule status --recursive | grep -c '^-' # must print 0 (all submodules
initialised)
```

Toolchain — standard path (requires working sudo):

```
Tools/setup/ubuntu.sh --no-nuttx --no-sim-tools
```

Toolchain — path actually used this session (no sudo available, D-002):

```
pip install --user cmake ninja
# build lcov/genhtml 2.0 from source: ./configure --prefix=$HOME/.local && make
install PREFIX=$HOME/.local
# install the DateTime Perl module via a user-local local::lib + cpan, not system-wide
echo 'export PATH=$HOME/.local/bin:$PATH' >> ~/.bashrc
```

Baseline build and test (P02):

```
make tests TESTFILTER=__no_tests__ # build only
make tests # build + run (config px4_sitl_test, ctest)
```

Coverage pipeline (P03–P10), via the project’s own wrapper (never upstream make tests_coverage):

```
tools/sqe_coverage.sh build
tools/sqe_coverage.sh baseline # pre-student-test capture
make tests PX4_CMAKE_BUILD_TYPE=Coverage TESTFILTER=__no_tests__ # switch build dir
to Coverage
tools/sqe_run_tests.sh build # build all Sqe* test binaries
tools/sqe_run_tests.sh run # ctest -R Sqe
tools/sqe_run_tests.sh shuffle # --gtest_shuffle --gtest_repeat=5|10 per
binary
tools/sqe_run_tests.sh probes # --gtest_also_run_disabled_tests
tools/sqe_coverage.sh student <tag> # student-suite-only capture
tools/sqe_coverage.sh final <tag> # full final capture (HTML + .info)
```

MC/DC and traceability checks (P06, P12):

```
python3 tools/sqe_mcdc_check.py work/mcdc/mcdc_matrix.csv
python3 tools/sqe_trace_check.py --px4 PX4-Autopilot --inventory work/inventory/
test_inventory.csv \
--mcdc work/mcdc/mcdc_matrix.csv
```

Workbook and report:

```
python3 tools/sqe_workbook.py # generates deliverables/<BASE>.xlsx
python3 tools/sqe_word_count.py deliverables/report/REPORT.md
```

Appendix B — Evidence index

Path	What it is
evidence/env/environment.md	tool/OS/compiler versions
evidence/baseline/summary.txt	baseline make tests result (147/147)
work/scope/SCOPE_RECORD.md, work/scope/CANDIDATES.md	scope selection and rejected candidates
work/basis/SETUP_MAP.md, work/basis/CFG_*.md, work/basis/INVENTORY_*.md	test-basis derivation
work/mcdc/MCDC_ANALYSIS.md, work/mcdc/mcdc_matrix.csv, work/mcdc/mcdc_check_report.md	MC/DC derivation and checker confirmation
evidence/coverage/baseline/per_file.md	baseline coverage
evidence/coverage/final/per_file.md, evidence/coverage/final/html/	final coverage (table + HTML)
evidence/coverage/compare_baseline_final.md	baseline vs final comparison
work/coverage_iterations.md	IT-0..IT-3 iteration history
work/GAPS.md	7 gap clusters with proofs
evidence/coverage/pertest/MC21_03_EVIDENCE.md, MC21_evidence.info, MC01_evidence.info	O3 structural-coverage evidence for the K pair
work/findings/REPORT_BLOCKS.md, work/FINDINGS.md	findings register and report-ready text
evidence/tests/xml/*.xml	per-binary GTest XML results
evidence/tests/logs/*_shuffle.log	shuffle/repeat stability logs
evidence/tests/probes/*	disabled-probe execution evidence
work/inventory/test_inventory.csv	full test-to-decision mapping (116 rows)
deliverables/24i3015_24i3166_24i3158_B.xlsx	testing workbook (2 sheets)
work/ai_assistance_log.md	full AI-assistance log
work/STATUS.md	phase/gate table, HUMAN-DECISION items
work/DECISIONS.md	ADR-lite decision log (D-002, D-004, D-008, D-009, D-010 cited above)